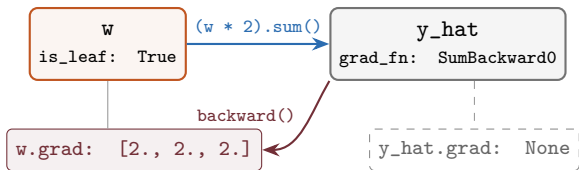


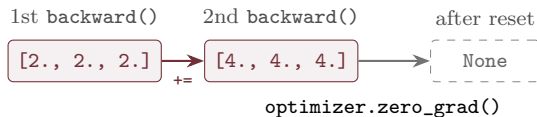
(a) Rule 1: only leaves keep `.grad`

```
w = torch.randn(3, requires_grad=True)
y_hat = (w * 2).sum()
y_hat.backward()
```



(b) Rule 2: `backward()` adds into `.grad`

The same `w` and `loss` as in (a):



Without the reset, a step follows the sum of all gradients since the last reset, not the current one.

(c) Rule 3: update in place, off the tape

✓ with `torch.no_grad():`
`w -= lr * w.grad` same leaf, new values

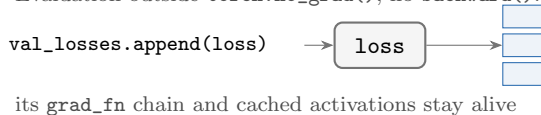
✗ `w = w - lr * w.grad` new non-leaf: next `w.grad` is `None`

✗ `w -= lr * w.grad` `RuntimeError: leaf used in an in-place op`

✗ with `torch.no_grad():`
`w = w - lr * w.grad` `requires_grad=False`
learning silently stops

(d) Rule 5: a kept tensor keeps its history

Evaluation outside `torch.no_grad()`, no `backward()`:



`val_losses.append(loss.item())`
a Python float: the graph can be freed

`val_losses.append(loss.detach())`
the same value with its history cut